



UE23EC343BC1 DEEP LEARNING

Dr. Rashmi N Ugarakhod

Department of Electronics and Communication Engineering

rashmin@pes.edu

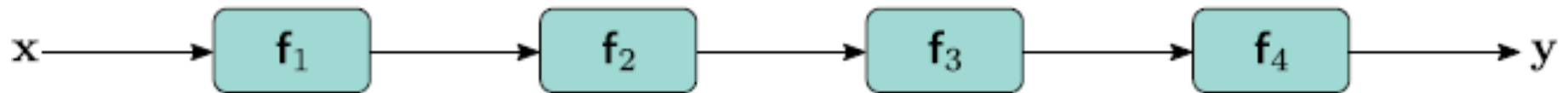
Residual Networks

Introduction

- *Residual blocks:* Each network layer computes an additive change to the current representation instead of transforming it directly.
- This allows deeper networks to be trained but causes an exponential increase in the activation magnitudes at initialization.
- Residual blocks employ *batch normalization* to compensate for this, which re-centers and rescales the activations at each layer.
- Residual blocks with batch normalization allow much deeper networks to be trained, and these networks improve performance across a variety of tasks.
- Architectures that combine residual blocks to tackle image classification, medical image segmentation, and human pose estimation are described.

Sequential Processing

Every network we have seen so far processes the data sequentially; each layer receives the previous layer's output and passes the result to the next:



For example, a three-layer network is defined by:

$$h_1 = f_1[x, \phi_1]$$

$$h_2 = f_2[h_1, \phi_2]$$

$$h_3 = f_3[h_2, \phi_3]$$

$$y = f_4[h_3, \phi_4]$$

where h_1 , h_2 , and h_3 denote the intermediate hidden layers, x is the network input, y

is the output, and the functions $f_k[\cdot, \phi_k]$ perform the processing.

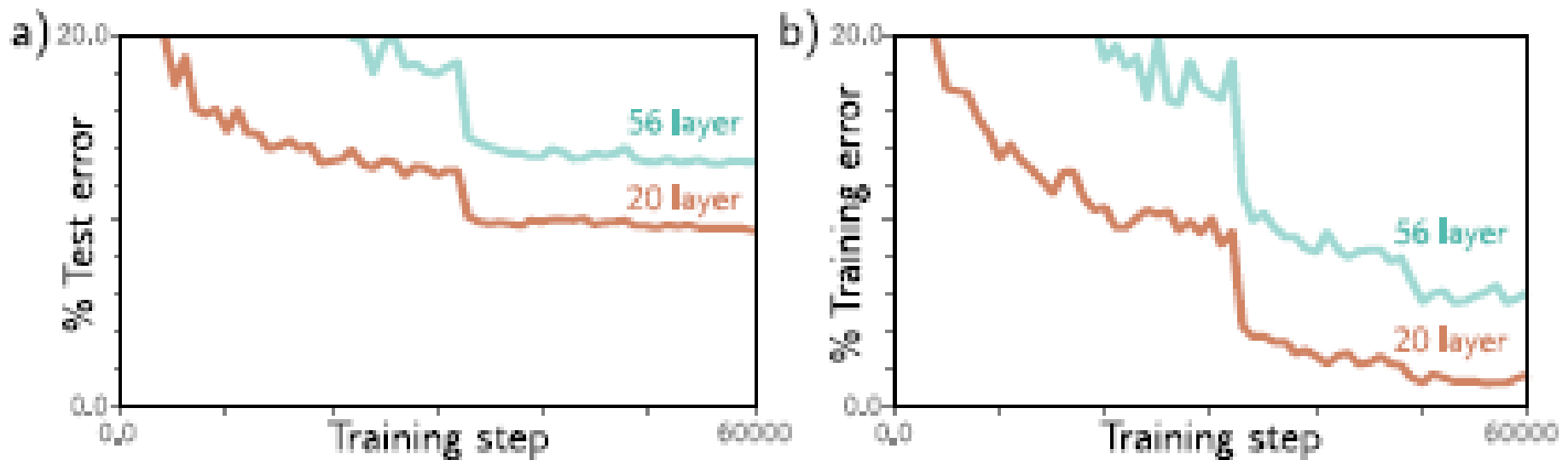
Sequential Processing

- In a standard neural network:
 - Each layer consists of a linear transformation followed by an activation function, and
 - The parameters ϕ_k comprise the weights and biases of the linear transformation.
- In a convolutional network:
 - Each layer consists of a set of convolutions followed by an activation function, and
 - The parameters comprise the convolutional kernels and biases.
- Since the processing is sequential, we can equivalently think of this network as a series of nested functions:

$$y = f_4[f_3[f_2[f_1[x, \phi_1], \phi_2], \phi_3], \phi_4]$$

Limitations of Sequential Processing

- In principle, adding more layers to a convolutional network does improve performance
- However, image classification performance decreases again as further layers are added.



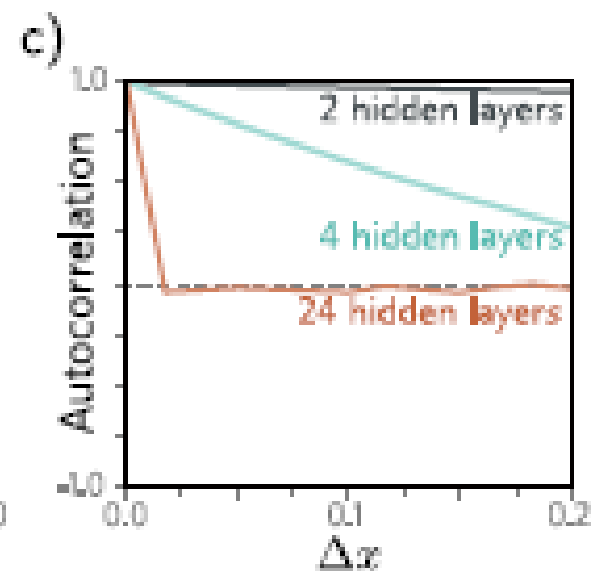
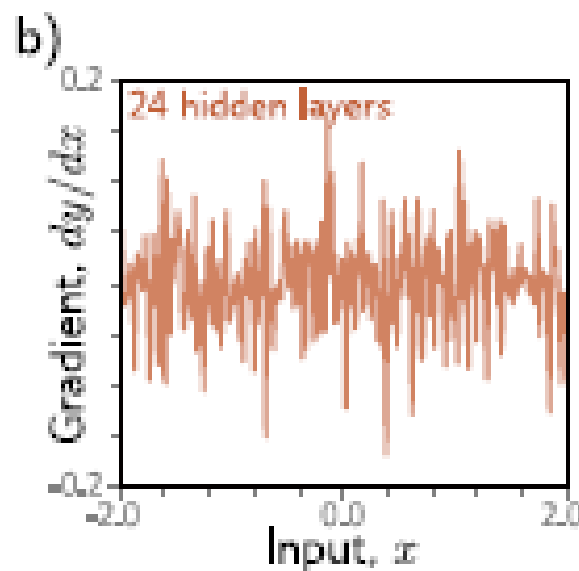
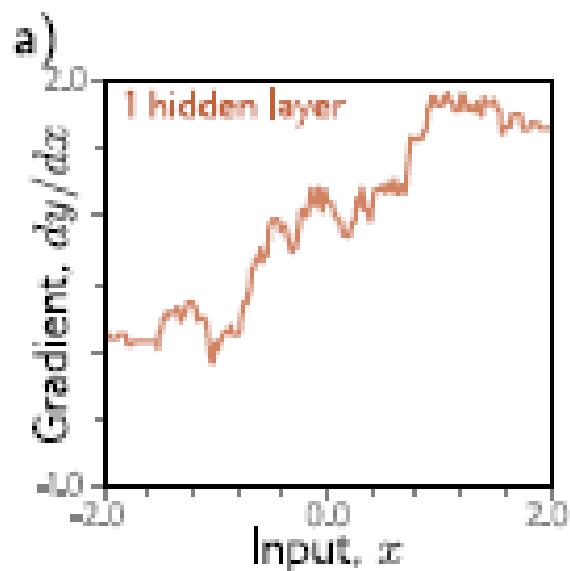
- This is surprising since models generally perform better as more capacity is added.
- Indeed, the decrease is present for both the training set and the test set, which implies that the problem is training deeper networks rather than the inability of deeper networks to generalize.
- This phenomenon is not completely understood.

Limitations of Sequential Processing

- One conjecture is that at initialization, the loss gradients change unpredictably when we modify parameters in early network layers.
- With appropriate initialization of the weights, the gradient of the loss with respect to these parameters will be reasonable (i.e., no exploding or vanishing gradients).
- However, the derivative assumes an infinitesimal change in the parameter, whereas optimization algorithms use a finite step size.
- Any reasonable choice of step size may move to a place with a completely different and unrelated gradient;
- The loss surface looks like an enormous range of tiny mountains rather than a single smooth structure that is easy to descend.
- Consequently, the algorithm doesn't make progress in the way that it does when the loss function gradient changes more slowly.
- This conjecture is supported by empirical observations of gradients in networks with a single input and output.

Limitations of Sequential Processing

- For a shallow network, the gradient of the output with respect to the input changes slowly as we change the input (figure a).
- However, for a deep network, a tiny change in the input results in a completely different gradient (figure b).
- This is captured by the autocorrelation function of the gradient (figure c).
- Nearby gradients are correlated for shallow networks, but this correlation quickly drops to zero for deep networks. This is termed the shattered gradients phenomenon.



Limitations of Sequential Processing

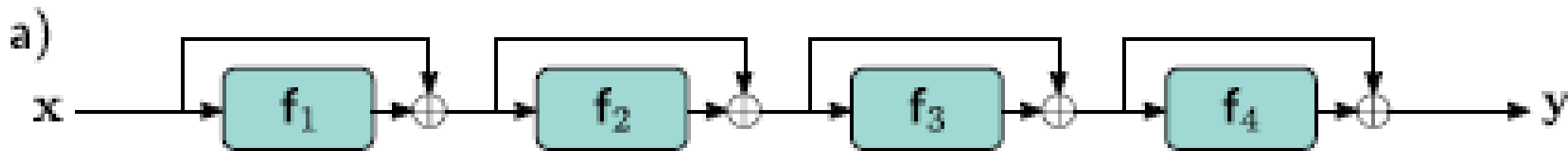
- Shattered gradients presumably arise because changes in early network layers modify the output in an increasingly complex way as the network becomes deeper.
- The derivative of the output y with respect to the first layer f_1 of the network is:

$$\frac{\partial y}{\partial f_1} = \frac{\partial f_4}{\partial f_3} \frac{\partial f_3}{\partial f_2} \frac{\partial f_2}{\partial f_1}$$

- When we change the parameters that determine f_1 , all of the derivatives in this sequence can change since layers f_2 , f_3 , and f_4 are themselves computed from f_1 .
- Consequently, the updated gradient at each training example may be completely different, and the loss function becomes badly behaved.

Residual connections and residual blocks

- Residual or skip connections are branches in the computational path, whereby the input to each network layer $f[\bullet]$ is added back to the output.



- By analogy, the residual network is defined as:

$$\begin{aligned}h_1 &= x + f_1[x, \phi_1] \\h_2 &= h_1 + f_2[h_1, \phi_2] \\h_3 &= h_2 + f_3[h_2, \phi_3] \\y &= h_3 + f_4[h_3, \phi_4]\end{aligned}$$

- Where the first term on the right-hand side of each line is the residual connection.

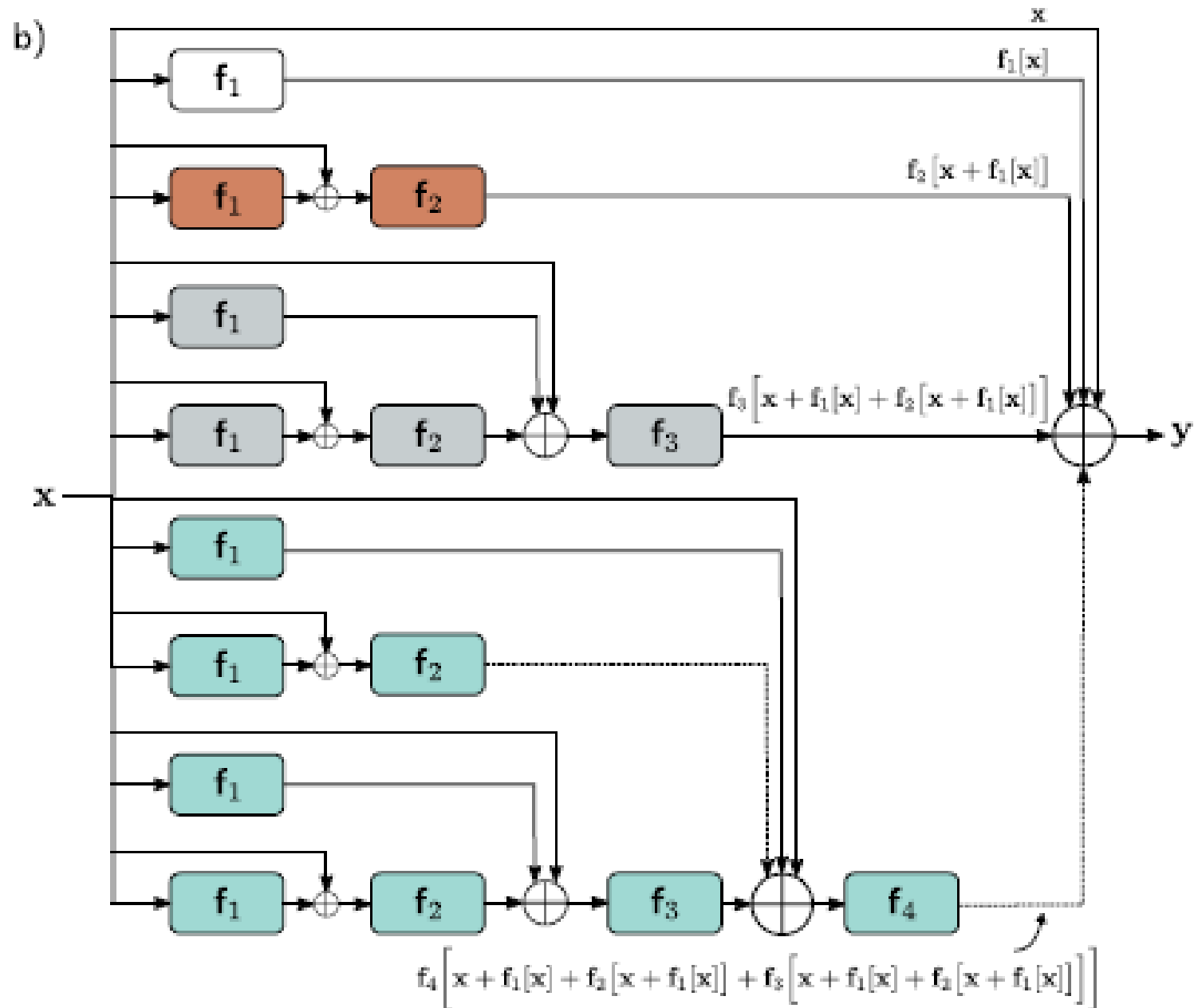
Residual connections and residual blocks

- Each function f_k learns an additive change to the current representation.
- It follows that their outputs must be the same size as their inputs.
- Each additive combination of the input and the processed output is known as a residual block or residual layer.
- The equations can be written as a single function by substituting in the expressions for the intermediate quantities h_k :

$$\begin{aligned} y = & x + f_1[x] \\ & + f_2[x + f_1[x]] \\ & + f_3 \left[x + f_1[x] + f_2[x + f_1[x]] \right] \\ & + f_4 \left[x + f_1[x] + f_2[x + f_1[x]] \right. \\ & \left. + f_3 \left[x + f_1[x] + f_2[x + f_1[x]] \right] \right] \end{aligned}$$

Residual connections and residual blocks

- We can think of this equation as “unraveling” the network (figure).



Residual connections and residual blocks

- Final network output is a sum of the input and four smaller networks, corresponding to each line of the equation;
- One interpretation is that residual connections turn the original network into an ensemble of these smaller networks whose outputs are summed to compute the result.
- A complementary way of thinking about this residual network is that it creates sixteen paths of different lengths from input to output.
- For example, the first function $f_1[x]$ occurs in eight of these sixteen paths, including as a direct additive term (i.e., a path length of one), and the analogous derivative is:

$$\frac{\partial y}{\partial f_1} = \mathbf{I} + \frac{\partial f_2}{\partial f_1} + \left(\frac{\partial f_3}{\partial f_1} + \frac{\partial f_3}{\partial f_2} \frac{\partial f_2}{\partial f_1} \right) + \left(\frac{\partial f_4}{\partial f_1} + \frac{\partial f_4}{\partial f_2} \frac{\partial f_2}{\partial f_1} + \frac{\partial f_4}{\partial f_3} \frac{\partial f_3}{\partial f_1} + \frac{\partial f_4}{\partial f_3} \frac{\partial f_3}{\partial f_2} \frac{\partial f_2}{\partial f_1} \right)$$

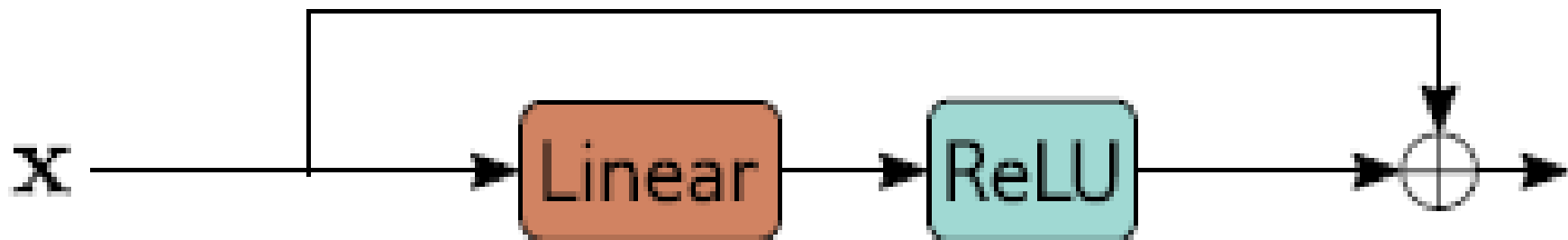
where there is one term for each of the eight paths.

Residual connections and residual blocks

- The identity term on the righthand side shows that changes in the parameters ϕ_1 in the first layer $f_1[x, \phi_1]$ contribute directly to changes in the network output y .
- They also contribute indirectly through the other chains of derivatives of varying lengths. In general, gradients through shorter paths will be better behaved.
- Since both the identity term and various short chains of derivatives will contribute to the derivative for each layer, networks with residual links suffer less from shattered gradients.

Order of operations in residual blocks

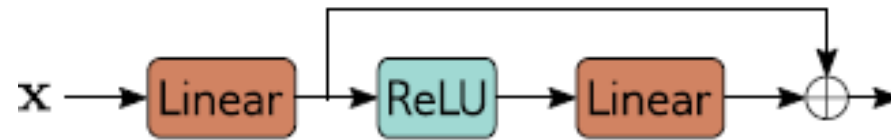
- The additive functions $f[x]$ could be any valid network layer (e.g., fully connected or convolutional).
- This is technically true, but the order of operations in these functions is important.
- They must contain a nonlinear activation function like a ReLU, or the entire network will be linear.
- However, in a typical network layer, the ReLU function is at the end, so the output is non-negative.



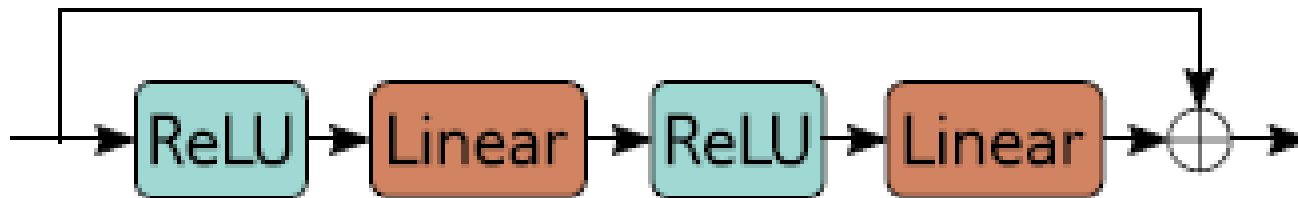
- If we adopt this convention, then each residual block can only increase the input values.

Order of operations in residual blocks

- Hence, it is typical to change the order of operations so that the activation function is applied first, followed by the linear transformation.



- Sometimes there may be several layers of processing within the residual block as shown in the figure, but these usually terminate with a linear transformation.



- Finally, we note that when we start these blocks with a ReLU operation, they will do nothing if the initial network input is negative since the ReLU will clip the entire signal to zero.
- Hence, it's typical to start the network with a linear transformation rather than a residual block.

Deeper networks with residual connections

- Adding residual connections roughly doubles the depth of a network that can be practically trained before performance degrades.
- However, we would like to increase the depth further.
- To understand why residual connections do not allow us to increase the depth arbitrarily, we must consider how the variance of the activation changes during the forward pass and how the gradient magnitudes change during the backward pass.

Exploding gradients in residual networks

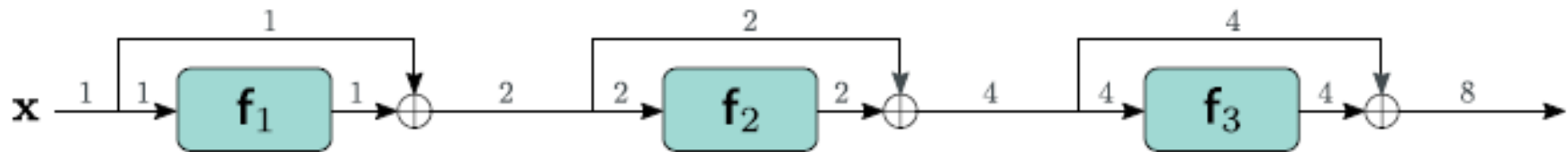
- Initializing the network parameters is critical.
- Without careful initialization, the magnitudes of the intermediate values during the forward pass of backpropagation can increase or decrease exponentially.
- Similarly, the gradients during the backward pass can explode or vanish as we move backward through the network.
- Hence, we initialize the network parameters so that the expected variance of the activations (in the forward pass) and gradients (in the backward pass) remains the same between layers.
- He initialization achieves this for ReLU activations by initializing the biases β to zero and choosing normally distributed weights Ω with mean zero and variance $2/D_h$ where D_h is the number of hidden units in the previous layer.

Exploding gradients in residual networks

- Now consider a residual network.
- We do not have to worry about the intermediate values or gradients vanishing with network depth since there exists a path whereby each layer directly contributes to the network output.
- However, even if we use He initialization within the residual block, the values in the forward pass increase exponentially as we move through the network.
- To see why, consider that we add the result of the processing in the residual block back to the input.
- Each branch has some (uncorrelated) variability. Hence, the overall variance increases when we recombine them.
- With ReLU activations and He initialization, the expected variance is unchanged by the processing in each block.

Exploding gradients in residual networks

- Consequently, when we recombine with the input, the variance doubles, growing exponentially with the number of residual blocks.



- This limits the possible network depth before floating point precision is exceeded in the forward pass.
- A similar argument applies to the gradients in the backward pass of the backpropagation algorithm.

Exploding gradients in residual networks

- Hence, residual networks still suffer from unstable forward propagation and exploding gradients even with He initialization.
- One approach that would stabilize the forward and backward passes would be to use He initialization and then multiply the combined output of each residual block by $1/\sqrt{2}$ to compensate for the doubling.
- However, it is more usual to use batch normalization.

Batch normalization

- Batch normalization or BatchNorm shifts and rescales each activation h so that its mean and variance across the batch B become values that are learned during training.
- First, the empirical mean m_h and standard deviation s_h are computed:

$$m_h = \frac{1}{|B|} \sum_{i \in B} h_i$$

$$s_h = \sqrt{\frac{1}{|B|} \sum_{i \in B} (h_i - m_h)^2},$$

where all quantities are scalars.

- Then we use these statistics to standardize the batch activations to have mean zero and unit variance:

$$h_i \leftarrow \frac{h_i - m_h}{s_h + \epsilon} \quad \forall i \in B,$$

Batch normalization

where ϵ is a small number that prevents division by zero if h_i is the same for every member of the batch and $s_h = 0$.

- Finally, the normalized variable is scaled by γ and shifted by δ :

$$h_i \leftarrow \gamma h_i + \delta \quad \forall i \in B.$$

- After this operation, the activations have mean δ and standard deviation γ across all members of the batch. Both of these quantities are learned during training.
- Batch normalization is applied independently to each hidden unit. In a standard neural network with K layers, each containing D hidden units, there would be KD learned offsets δ and KD learned scales γ .
- In a convolutional network, the normalizing statistics are computed over both the batch and the spatial position.
- If there were K BatchNorm layers, each containing C channels, there would be KC offsets and KC scales.

Batch normalization

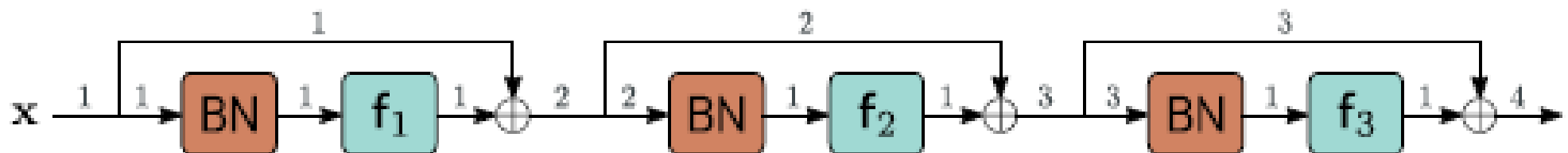
- At test time, we do not have a batch from which we can gather statistics.
- To resolve this, the statistics m_h and s_h are calculated across the whole training dataset (rather than just a batch) and frozen in the final network.

Costs and benefits of batch normalization

- Batch normalization makes the network invariant to rescaling the weights and biases that contribute to each activation;
- If these are doubled, then the activations also double, the estimated standard deviation sh doubles, and the normalization compensates for these changes.
- This happens separately for each hidden unit.
- Consequently, there will be a large family of weights and biases that all produce the same effect.
- Batch normalization also adds two parameters, γ and δ , at every hidden unit, which makes the model somewhat larger.
- Hence, it both creates redundancy in the weight parameters and adds extra parameters to compensate for that redundancy.
- This is obviously inefficient, but batch normalization also provides several benefits.

Costs and benefits of batch normalization

- **Stable forward propagation:**
- If we initialize the offsets δ to zero and the scales γ to one, then each output activation will have unit variance.
- In a regular network, this ensures the variance is stable during forward propagation at initialization.
- In a residual network, the variance must still increase as we add a new source of variation to the input at each layer.
- However, it will increase linearly with each residual block; the k th layer adds one unit of variance to the existing variance of k as shown:



Costs and benefits of batch normalization

- At initialization, this has the side-effect that later layers make a smaller change to the overall variation than earlier ones.
- The network is effectively less deep at the start of training since later layers are close to computing the identity.
- As training proceeds, the network can increase the scales γ in later layers and can control its own effective depth.
- **Higher learning rates:**
- Empirical studies and theory both show that batch normalization makes the loss surface and its gradient change more smoothly (i.e., reduces shattered gradients).
- This means we can use higher learning rates as the surface is more predictable. Higher learning rates improve test performance.

Costs and benefits of batch normalization

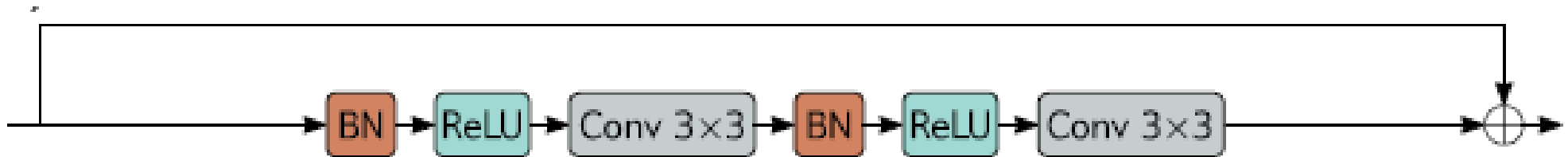
- **Regularization:**
- Adding noise to the training process can improve generalization.
- Batch normalization injects noise because the normalization depends on the batch statistics.
- The activations for a given training example are normalized by an amount that depends on the other members of the batch and will be slightly different at each training iteration.

Common residual architectures

- Residual connections are now a standard part of deep learning pipelines.
- Some well-known architectures that incorporate them are discussed:

ResNet

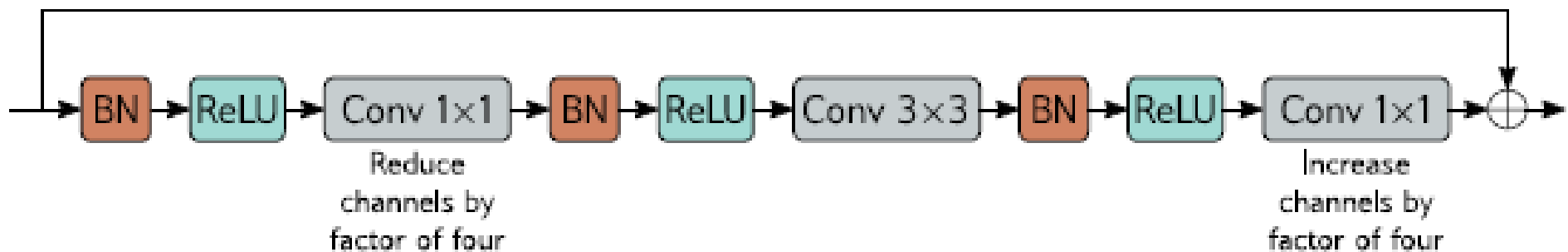
- Residual blocks were first used in convolutional networks for image classification.
- The resulting networks are known as residual networks, or ResNets for short.
- In ResNets, each residual block contains a batch normalization operation, a ReLU activation function, and a convolutional layer.
- This is followed by the same sequence again before being added back to the input (figure shown below).



- Trial and error have shown that this order of operations works well for image classification.

ResNet

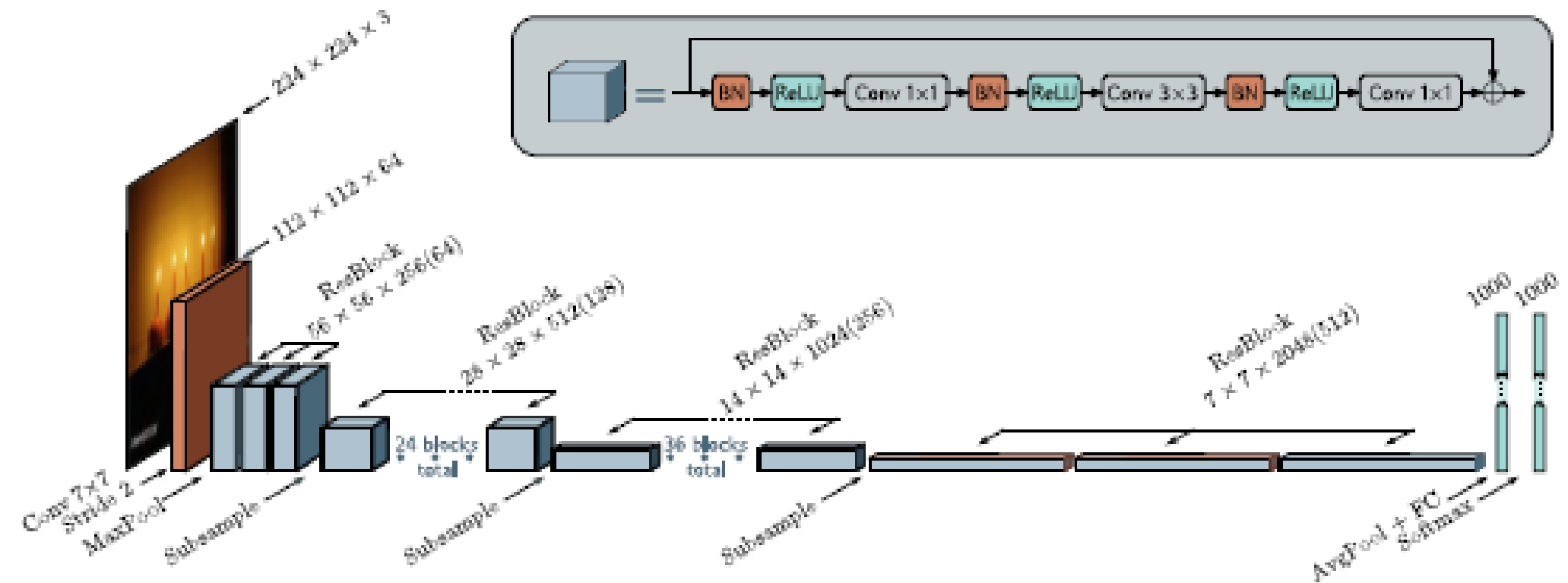
- For very deep networks, the number of parameters may become undesirably large.
- Bottleneck residual blocks make more efficient use of parameters using three convolutions.
- The first has a 1×1 kernel and reduces the number of channels.
- The second is a regular 3×3 kernel, and the third is another 1×1 kernel to increase the number of channels back to the original amount (figure shown below).



- In this way, we can integrate information over a 3×3 pixel area using fewer parameters.

ResNet

- The ResNet-200 model contains 200 layers and was used for image classification on the ImageNet database.



ResNet

- The architecture resembles AlexNet and VGG but uses bottleneck residual blocks instead of vanilla convolutional layers.
- As with AlexNet and VGG, these are periodically interspersed with decreases in spatial resolution and simultaneous increases in the number of channels.
- Here, the resolution is decreased by downsampling using convolutions with stride two.
- The number of channels is increased either by appending zeros to the representation or by using an extra 1×1 convolution.
- At the start of the network is a 7×7 convolutional layer, followed by a downsampling operation.
- At the end, a fully connected layer maps the block to a vector of length 1000. This is passed through a softmax layer to generate class probabilities.

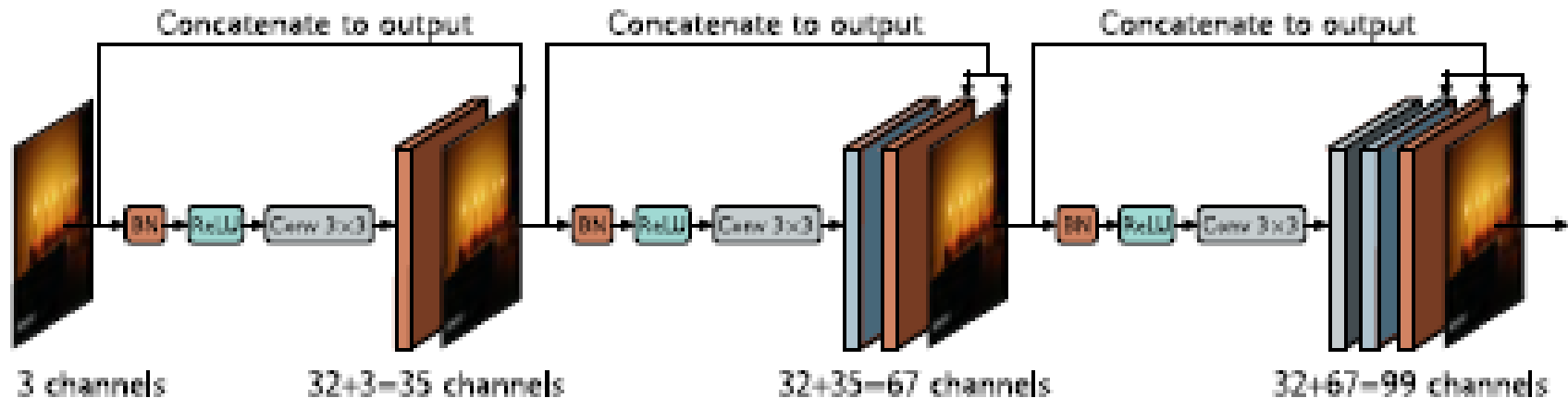
ResNet

- The ResNet-200 model achieved a remarkable 4.8% error rate for the correct class being in the top five and 20.1% for identifying the correct class correctly.
- This compared favorably with AlexNet (16.4%, 38.1%) and VGG (6.8%, 23.7%) and was one of the first networks to exceed human performance (5.1% for being in the top five guesses).
- However, this model was conceived in 2016 and is far from state-of-the-art. At the time of writing, the best-performing model on this task has a 9.0% error for identifying the class correctly.
- This and all the other current top-performing models for image classification are now based on transformers.

DenseNet

- Residual blocks receive the output from the previous layer, modify it by passing it through some network layers, and add it back to the original input.
- An alternative is to concatenate the modified and original signals. This increases the representation size (in terms of channels for a convolutional network), but an optional subsequent linear transformation can map back to the original size (a 1×1 convolution for a convolutional network).
- This allows the model to add the representations together, take a weighted sum, or combine them in a more complex way.
- The DenseNet architecture uses concatenation so that the input to a layer comprises the concatenated outputs from all previous layers (shown in figure).

DenseNet



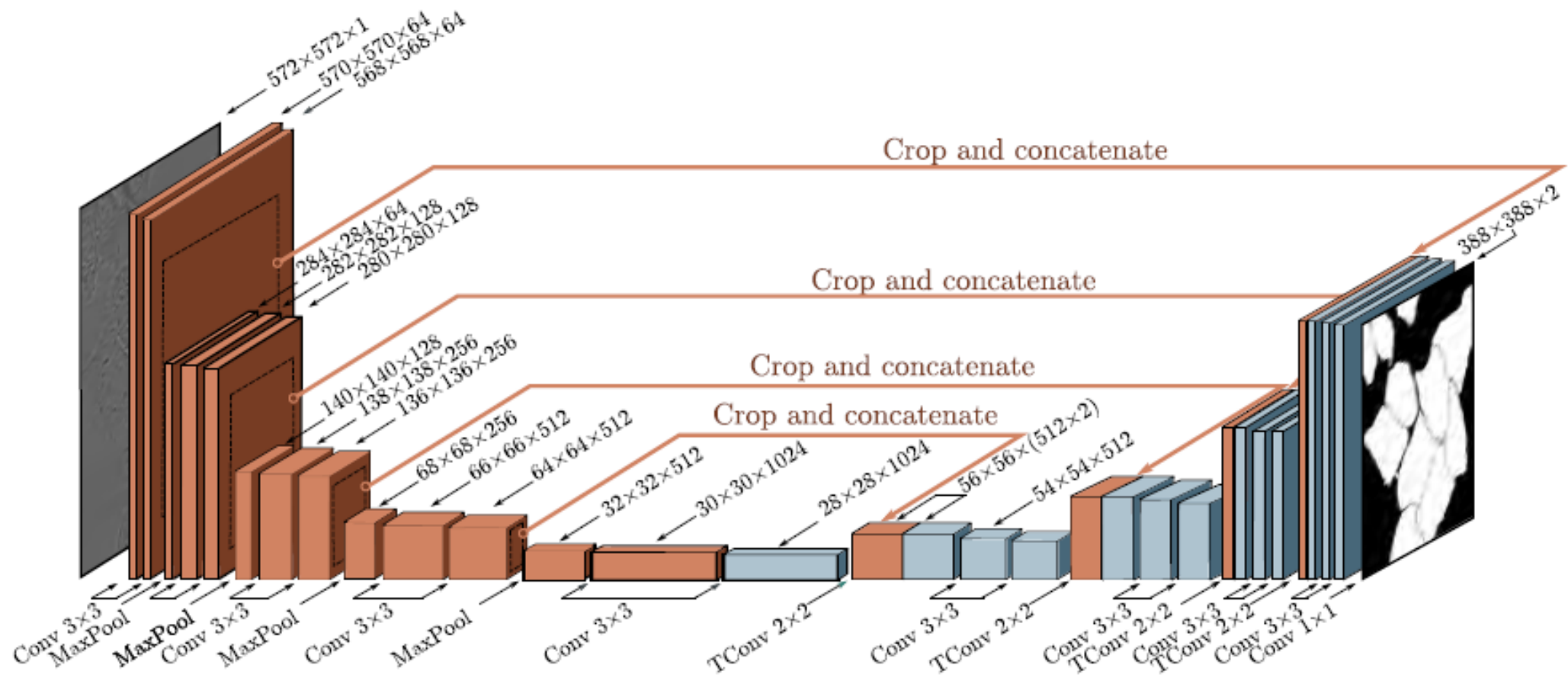
- These are processed to create a new representation that is itself concatenated with the previous representation and passed to the next layer.
- This concatenation means there is a direct contribution from earlier layers to the output, so the loss surface behaves reasonably.
- In practice, this can only be sustained for a few layers because the number of channels (and hence the number of parameters required to process them) becomes increasingly large.

DenseNet

- This problem can be alleviated by applying a 1×1 convolution to reduce the number of channels before the next 3×3 convolution is applied.
- In a convolutional network, the input is periodically downsampled.
- Concatenation across the downsampling makes no sense since the representations have different sizes.
- Consequently, the chain of concatenation is broken at this point, and a smaller representation starts a new chain.
- In addition, another bottleneck 1×1 convolution can be applied when the downsampling occurs to control the representation size further.
- This network performs competitively with ResNet models on image classification; indeed, it can perform better for a comparable parameter count.
- This is presumably because it can reuse processing from earlier layers more flexibly.

U-Nets and hourglass networks

- The U-Net (shown in figure below) is an encoder-decoder architecture where the earlier representations are concatenated to the later ones.



U-Nets and hourglass networks

- The original implementation used “valid” convolutions, so the spatial size decreases by two pixels each time a 3×3 convolutional layer is applied.
- This means that the upsampled version is smaller than its counterpart in the encoder, which must be cropped before concatenation.
- Subsequent implementations have used zero padding, where this cropping is unnecessary.
- Note that the U-Net is completely convolutional, so after training, it can be run on an image of any size.

U-Nets and hourglass networks

- The U-Net was intended for segmenting medical images (figure shown below) but has found many other uses in computer graphics and vision.

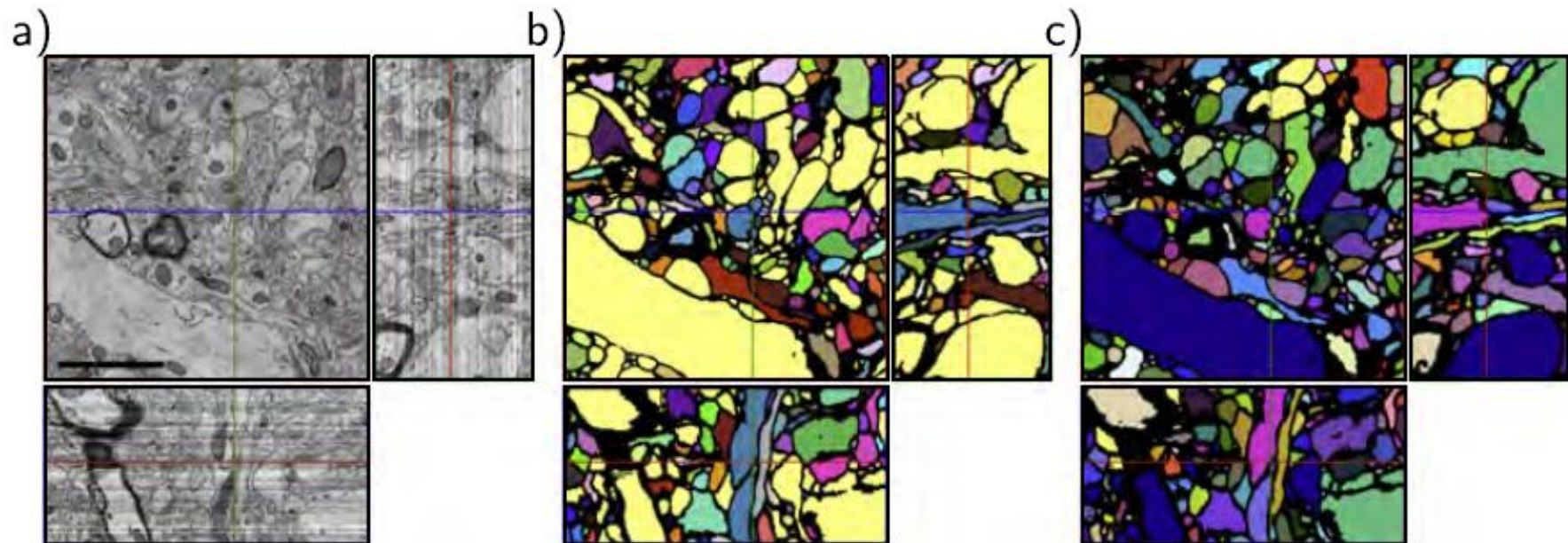
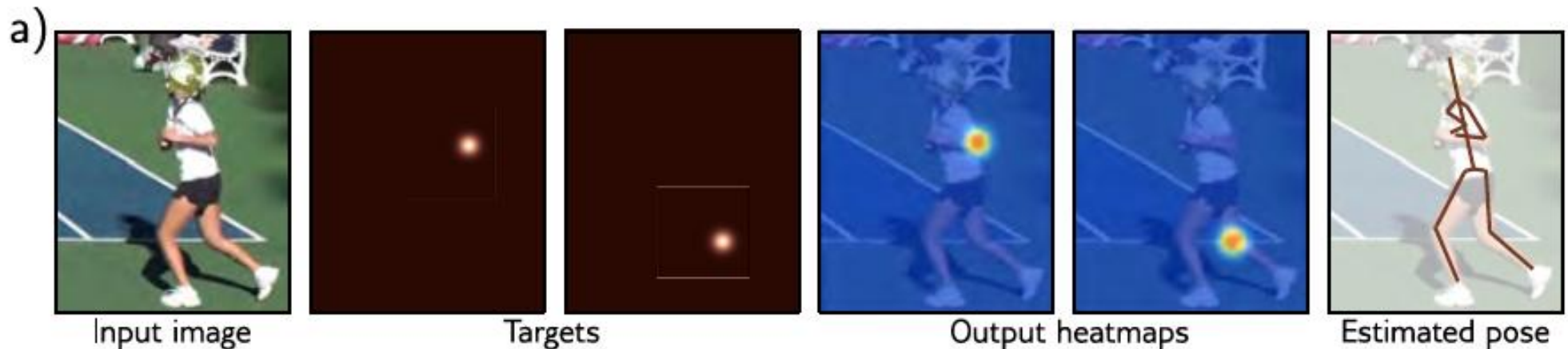


Figure 11.11 Segmentation using U-Net in 3D. a) Three slices through a 3D volume of mouse cortex taken by scanning electron microscope. b) A single U-Net is used to classify voxels as being inside or outside neurites. Connected regions are identified with different colors. c) For a better result, an ensemble of five U-Nets is trained, and a voxel is only classified as belonging to the cell if all five networks agree. Adapted from Falk et al. (2019).

U-Nets and hourglass networks

- Hourglass networks are similar but apply further convolutional layers in the skip connections and add the result back to the decoder rather than concatenating it.
- A series of these models form a stacked hourglass network that alternates between considering the image at local and global levels.
- Such networks are used for pose estimation (figure shown below).



- The system is trained to predict one “heatmap” for each joint, and the estimated position is the maximum of each heatmap.

U-Nets and hourglass networks

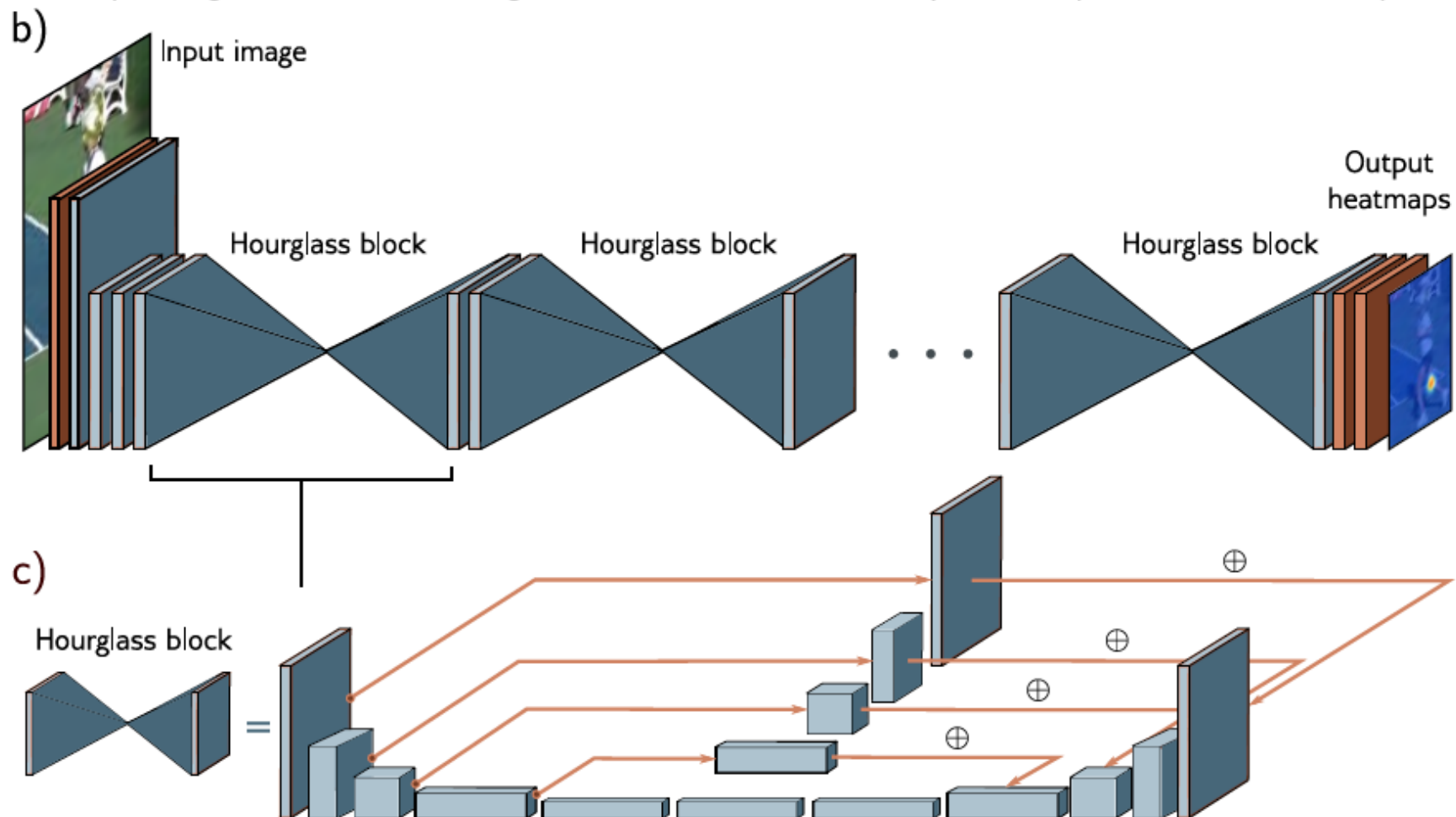


Figure 11.12 Stacked hourglass networks for pose estimation. a) The network input is an image containing a person, and the output is a set of heatmaps, with one heatmap for each joint. This is formulated as a regression problem where the targets are heatmap images with small, highlighted regions at the ground-truth joint positions. The peak of the estimated heatmap is used to establish each final joint position. b) The architecture consists of initial convolutional and residual layers followed by a series of hourglass blocks. c) Each hourglass block consists of an encoder-decoder network similar to the U-Net except that the convolutions use zero padding, some further processing is done in the residual links, and these links add this processed representation rather than concatenate it. Each blue cuboid is itself a bottleneck residual block (figure 11.7b). Adapted from Newell et al. (2016).

Why do nets with residual connections perform so well?

- Residual networks allow much deeper networks to be trained; it's possible to extend the ResNet architecture to 1000 layers and still train effectively.
- The improvement in image classification performance was initially attributed to the additional network depth, but two pieces of evidence contradict this viewpoint.
- First, shallower, wider residual networks sometimes outperform deeper, narrower ones with a comparable parameter count.
- In other words, better performance can sometimes be achieved with a network with fewer layers but more channels per layer.
- Second, there is evidence that the gradients during training do not propagate effectively through very long paths in the unraveled network.

Why do nets with residual connections perform so well?

- In effect, a very deep network may act more like a combination of shallower networks.
- The current view is that residual connections add some value of their own, as well as allowing deeper networks to be trained.
- This perspective is supported by the fact that the loss surfaces of residual networks around a minimum tend to be smoother and more predictable than those for the same network when the skip connections are removed (as shown in figure).

Why do nets with residual connections perform so well?

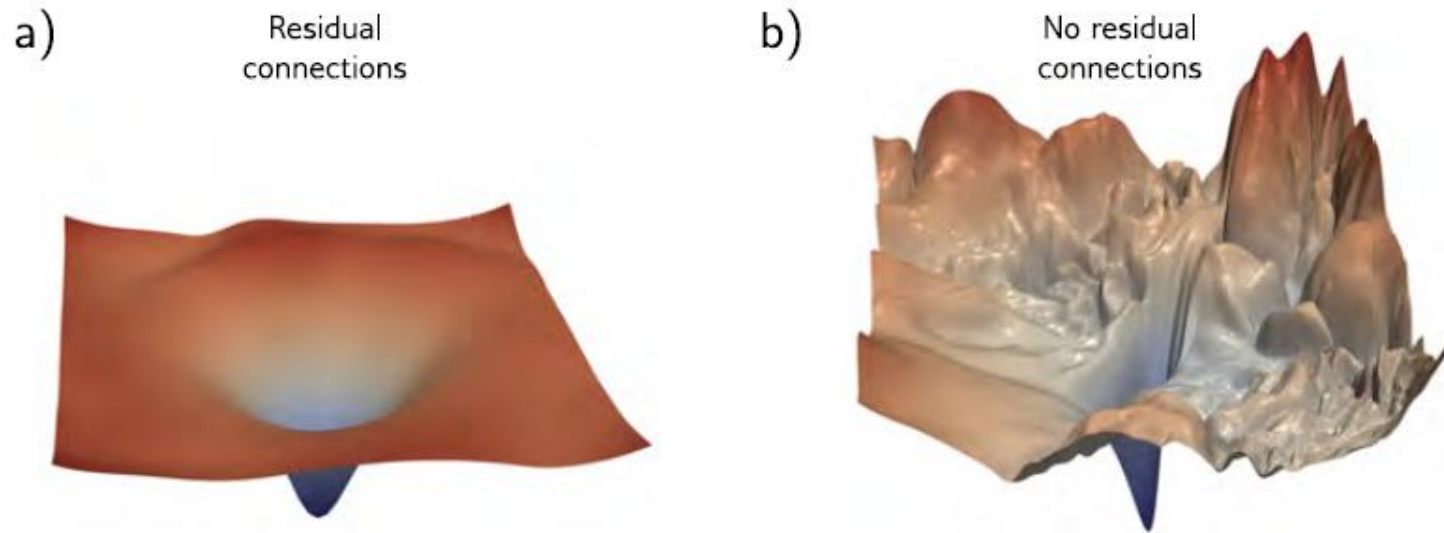


Figure 11.13 Visualizing neural network loss surfaces. Each plot shows the loss surface in two random directions in parameter space around the minimum found by SGD for an image classification task on the CIFAR-10 dataset. These directions are normalized to facilitate side-by-side comparison. a) Residual net with 56 layers. b) Results from the same network without skip connections. The surface is smoother with the skip connections. This facilitates learning and makes the final network performance more robust to minor errors in the parameters, so it will likely generalize better. Adapted from Li et al. (2018b).

This may make it easier to learn a good solution that generalizes well.